



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

ASSIGNMENT 1

Subject Name: Exploratory Data Analysis

Subject Code: BAI515E

Submitted By:

Amrutha(1AY24AI007)

Submitted To:

Ms. Surbhi



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

Table of Contents

Sl. No.	Topic	PAGE NO
1	Introduction	3
2	Data Generation	4-5
3	Pairwise Squared Distances	6-7
4	Verification of Distance Matrix	8
5	Sorting for Nearest Neighbors	9-10
6	Visualization of Neighbor Network	11-12
7	Performance Comparison	13-16
8	Conclusion	17



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

1. INTRODUCTION

I worked with a set of points on a two-dimensional plane and found the nearest neighbors of each point using NumPy.

The main purpose of this assignment is to understand how NumPy broadcasting can be used to perform calculations on many points at the same time. Instead of calculating the distance between every pair of points using loops, broadcasting allows us to calculate all the distances together.

I also used `argsort()` and `argpartition()` to find the nearest neighbors. Finally, I compared the performance of the NumPy method with a loop-based method.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

2. DATA GENERATION

First, I generated 10 random points using NumPy. Each point has two values, representing its X-coordinate and Y-coordinate.

I used a fixed random seed so that the same points are generated every time the program is executed.

Code

```
import numpy as np

import matplotlib.pyplot as plt

# Fixed random seed

np.random.seed(42)

# Generate 10 random points

X = np.random.rand(10, 2)

print("Points:")

print(X)

print("Shape of X:", X.shape)

# Plot the points

plt.scatter(X[:, 0], X[:, 1])

plt.xlabel("X-coordinate")

plt.ylabel("Y-coordinate")

plt.title("10 Random Points")

plt.show()
```

Explanation

The function `np.random.rand(10, 2)` creates an array containing 10 points and 2 coordinates for each point.

Therefore, the shape of the array is: (10, 2)

The scatter plot shows the location of all 10 points on the two-dimensional plane.

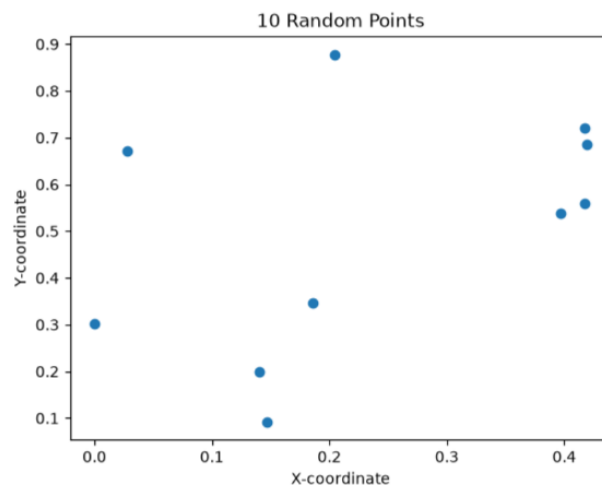


Figure 1: Plot of 10 Random Points



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

3. PAIRWISE SQUARED DISTANCES

Next, I calculated the squared Euclidean distance between every pair of points.

For two points (x_1, y_1) and (x_2, y_2) , the squared Euclidean distance is:

$$(x_1 - x_2)^2 + (y_1 - y_2)^2$$

I used NumPy broadcasting so that the distances between all pairs of points could be calculated without using an explicit loop.

Single-line calculation(CODE)

```
D2 = np.sum((X[:, np.newaxis, :] - X[np.newaxis, :, :])**2, axis=2)
```

```
print("Squared distance matrix:")
```

```
print(D2)
```

The result is a **10 × 10 distance matrix**.

Each element of the matrix represents the squared distance between two points.

3.1 Coordinate Differences

The first step is to find the difference between the coordinates of every pair of points.

Code:

```
diff = X[:, np.newaxis, :] - X[np.newaxis, :, :]
```

```
print("Coordinate differences shape:", diff.shape)
```

Output :

```
Coordinate differences shape: (10, 10, 2)
```

There are 10 points compared with 10 points, and each comparison has 2 coordinate differences.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

3.2 Squared Differences

Next, I squared the coordinate differences.

Code:

```
sq_diff = diff ** 2  
  
print("Squared differences shape:", sq_diff.shape)
```

Output

Squared differences shape: (10, 10, 2)

3.3 Summed Squared Distances

Finally, I added the two squared coordinate differences.

Code

```
D2 = np.sum(sq_diff, axis=2)  
  
print("Summed squared distances shape:", D2.shape)
```

Output

Summed squared distances shape: (10, 10)

This gives the final squared-distance matrix.

Table 1: Shapes of Intermediate Results

Step	Result	Shape
Coordinate differences	diff	(10, 10, 2)
Squared differences	sq_diff	(10, 10, 2)
Summed squared distances	D2	(10, 10)



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

4. VERIFICATION OF DISTANCE MATRIX

I checked the diagonal values of the distance matrix to make sure they are all zero.

Code

```
diagonal = np.diag(D2)
print("Diagonal:")
print(diagonal)
print("Are all diagonal values zero?",
      np.all(diagonal == 0))
```

Output

```
Diagonal:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
Are all diagonal values zero? True
```

Explanation

The diagonal represents the distance of every point from itself.

For example, the distance between point 0 and point 0 is zero. Similarly, the distance between point 1 and point 1 is zero.

Therefore, all the diagonal values must be zero.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

5. SORTING FOR NEAREST NEIGHBORS

5.1 Using argsort()

I used `np.argsort()` to arrange the points according to their distances.

Code

```
neighbor_order = np.argsort(D2, axis=1)
print("Neighbor ordering:")
print(neighbor_order)
```

`argsort()` returns the indices in sorted order.

The first value in each row is the point itself because the distance from a point to itself is zero.

Therefore, the first column contains:

0 1 2 3 4 5 6 7 8 9

For example, the first value in row 0 is 0 because point 0 has zero distance from itself.

5.2 Finding $K = 2$ Nearest Neighbors

In this assignment, $K = 2$.

Since each point itself is included in the distance matrix, I first find $K + 1$, which means 3 points.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

Code

```
K = 2
```

```
nearest = np.argpartition(D2, K, axis=1)[:K+1]
```

```
print("K+1 nearest points:")
```

```
print(nearest)
```

Then I remove the point itself:

```
nearest_neighbors = np.array([
```

```
    row[row != i][:K]
```

```
    for i, row in enumerate(nearest) ])
```

```
print("2 nearest neighbors:")
```

```
print(nearest_neighbors)
```

Why use argpartition()?

argsort() completely sorts all the distances. But here, I only need the two nearest neighbors.

argpartition() places the required smallest values at the beginning without completely sorting all the values. Therefore, it can be more efficient when only a small number of nearest neighbors are required.

6. VISUALIZATION OF NEIGHBOR NETWORK

After finding the two nearest neighbors of each point, I plotted the points again and connected each point to its two nearest neighbors.

Code

```
plt.figure(figsize=(8, 6))

# Plot the points

plt.scatter(X[:, 0], X[:, 1])

# Add point numbers

for i in range(len(X)):

    plt.text(X[i, 0], X[i, 1], str(i))

# Draw lines to nearest neighbors

for i in range(len(X)):

    for j in nearest_neighbors[i]:

        plt.plot(

            [X[i, 0], X[j, 0]],

            [X[i, 1], X[j, 1]] )

plt.xlabel("X-coordinate")

plt.ylabel("Y-coordinate")

plt.title("2-Nearest Neighbor Network")

plt.show()
```

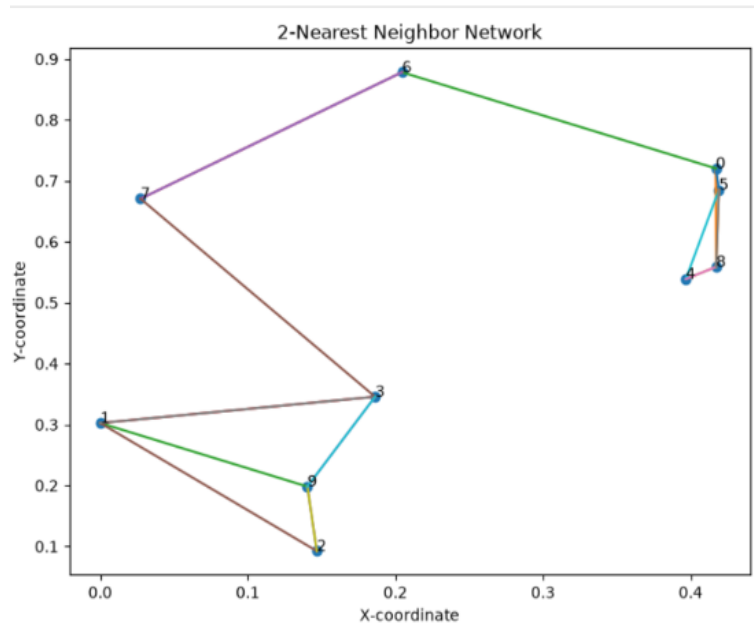


Figure 2: 2-Nearest Neighbor Network

Explanation

The points represent the original data points and the lines represent the nearest-neighbor connections.

Some points may have more than two lines connected to them. This happens because several different points can have the same point as one of their nearest neighbors.

For example, if points 1, 3 and 5 all have point 2 as a nearest neighbor, point 2 will have more than two connections.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

7. PERFORMANCE COMPARISON

In this part, I compared the loop-based method with the vectorized NumPy method.

7.1 Loop-Based Method

```
def loop_nearest_neighbors(X, K=2):
```

```
    N = len(X)
```

```
    result = []
```

```
    for i in range(N):
```

```
        distances = []
```

```
        for j in range(N):
```

```
            distance = np.sum((X[i] - X[j]) ** 2)
```

```
            distances.append(distance)
```

```
        order = np.argsort(distances)
```

```
        neighbors = [x for x in order if x != i][:K]
```

```
        result.append(neighbors)
```



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

```
return np.array(result)
```

7.2 Vectorized Method

```
def vectorized_nearest_neighbors(X, K=2):
```

```
    D2 = np.sum(  
        (X[:, np.newaxis, :] -  
         X[np.newaxis, :, :]) ** 2,  
        axis=2 )
```

```
    nearest = np.argpartition(  
        D2, K, axis=1
```

```
    )[:, :K+1]
```

```
    result = np.array([  
        row[row != i][:K]  
        for i, row in enumerate(nearest)  
    ])
```

```
    return result
```

7.3 Timing Both Methods

I used time.time() to measure the execution time for 100, 1000 and 5000 points.

Code

```
import time
```

```
for N in [100, 1000, 5000]:
```



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

```
X_test = np.random.rand(N, 2)

# Loop-based method
start = time.time()

loop_nearest_neighbors(X_test)

end = time.time()

loop_time = end - start

# Vectorized method
start = time.time()

vectorized_nearest_neighbors(X_test)

end = time.time()

vectorized_time = end - start

print("\nN =", N)

print("Loop-based time =", loop_time, "seconds")

print("Vectorized time =", vectorized_time, "seconds")
```



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

Performance Table

Table 2: Performance Comparison

Number of Points (N)	Loop-Based Time (seconds)	Vectorized Time (seconds)
100	188 ms $\pm$ 6.39 ms	2.14 ms $\pm$ 48.6 μ s
1,000	18.1 s $\pm$ 189 ms	102 ms $\pm$ 1.94 ms
5,000	1h 4min 28s $\pm$ 2h 3min 47s	1.96 s $\pm$ 78.7 ms

Discussion

From the comparison, I observed that the execution time increases as the number of points increases.

The vectorized method can be faster because NumPy performs operations on complete arrays instead of repeatedly performing calculations using Python loops. Broadcasting also allows many distance calculations to be performed together.

The main advantage is that the distance calculation code does not need to be rewritten when the number of points changes.



ACHARYA INSTITUTE OF TECHNOLOGY

Affiliated to Visvesvaraya Technological University, Belagavi, Govt. of Karnataka.

Approved by AICTE, New Delhi

Department of Artificial Intelligence & Machine Learning

8. CONCLUSION

In this assignment, I learned how NumPy broadcasting can be used to calculate distances between multiple points efficiently.

I first generated 10 random two-dimensional points and calculated their pairwise squared distances. I then checked the diagonal of the distance matrix and confirmed that all the values were zero.

After that, I used `argsort()` to obtain the ordering of the nearest points and `argpartition()` to find the two nearest neighbors. I also created a plot showing the connections between the points.

Finally, I compared a normal loop-based method with the vectorized NumPy method for different numbers of points. This helped me understand why vectorized operations are useful when working with larger datasets.

Overall, this assignment gave me a basic understanding of **NumPy broadcasting, distance calculation, sorting, nearest neighbors and performance comparison.**